Personal Finance Assistant — System Documentation

Oct 25, 2026 · @weam

1. Overview

The Personal Finance Assistant is a working prototype that helps a student or young earner in Sudan understand their money, plan their savings and ask questions about it in Arabic or English. A language model holds the conversation, while Python does all the arithmetic and a small verified knowledge base supplies the facts. This is the design the research proposal sets out.

Who it is for: university students and young earners managing one income and a few savings goals. Amounts are in Sudanese pounds (SDG, ج.س).

What it does

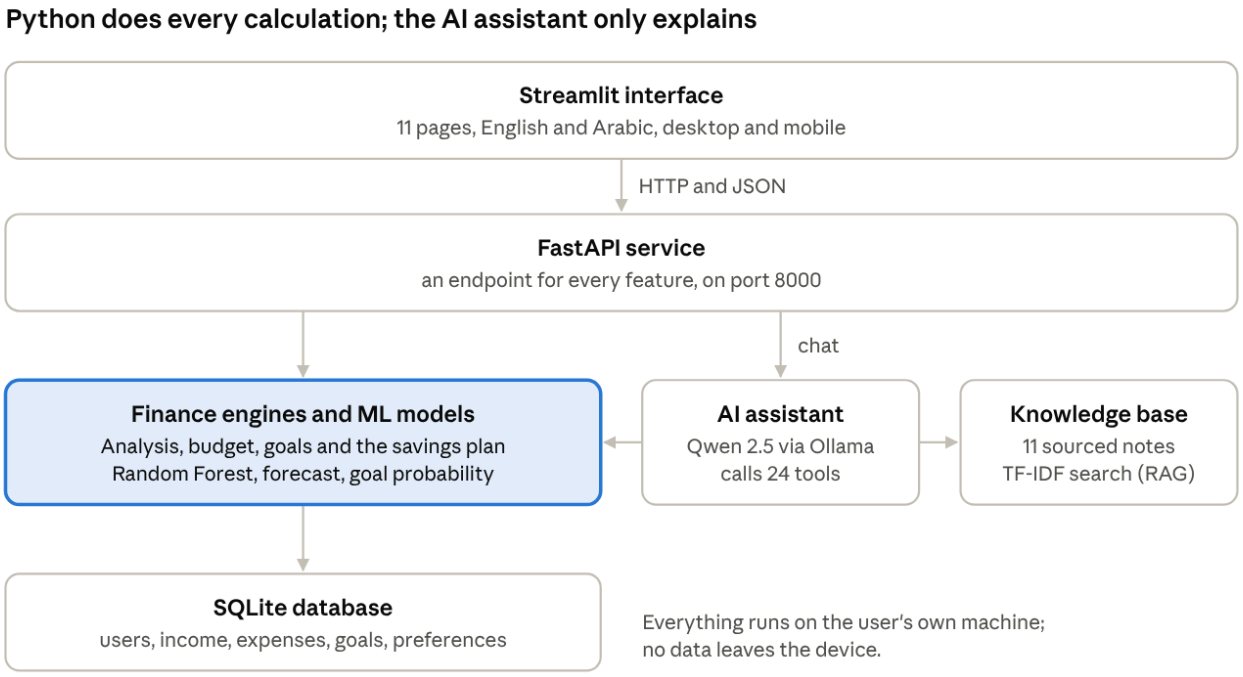
Area	What the user gets
Track	Expenses, income and recurring bills, entered by hand or imported from a CSV file
Understand	The month in one sentence, spending by category and subcategory, trends, income stability
Predict	Next month's spending as an expected value with an uncertainty range
Decide	"Can I afford it?": the chance of reaching an amount by a date, a Likely / Borderline / Unlikely label and a concrete action
Plan	A step-by-step plan that closes the savings gap, chosen by a Random Forest and sized by the engine; spending marked essential is never cut
Explore	What-if scenarios, and a suggested order for several goals
Ask	A local AI assistant (Qwen) that answers only from tool results and cited sources, and says when it cannot answer
Learn	Money tips from official sources (CFPB, FDIC, IMF, Duke University) and an interest calculator

Scope: one person per profile, data kept on the device (SQLite and a local model), synthetic demo data, Arabic and English with a right-to-left layout, and desktop and mobile browsers. The assistant is educational, not licensed financial advice.

Built with: Python 3, FastAPI, Streamlit, SQLite, scikit-learn, Ollama running Qwen 2.5 (3B), and Altair charts.

2. Architecture

The system has four layers. The language model never calculates: it calls Python tools for every number, and the knowledge base for every general fact.



System architecture · 4 layers

A request runs top-down. The interface calls the API. The API runs the engines, which read SQLite, or passes a chat message to the assistant. The assistant calls the same engines through its tools and searches the knowledge base for general questions.

Layer	Technology	Role
Interface	Streamlit 1.50+, Altair, streamlit-sortables	Pages, charts, drag-and-drop boards, Arabic right-to-left layout
API	FastAPI, Uvicorn, Pydantic	One endpoint per feature, input validation
Engines	Python, pandas	Analysis, budget, goals, recommendation plan, scenarios, onboarding, CSV import
Machine learning	scikit-learn, SciPy	Random Forest (which actions), LinearRegression forecast, goal probability
Assistant	Ollama + Qwen 2.5 3B	Understands the question, calls tools, explains the results
Knowledge	TF-IDF vectors (scikit-learn)	Retrieves verified passages with their sources
Storage	SQLite	One local file, `data/finance.db`

Folder structure

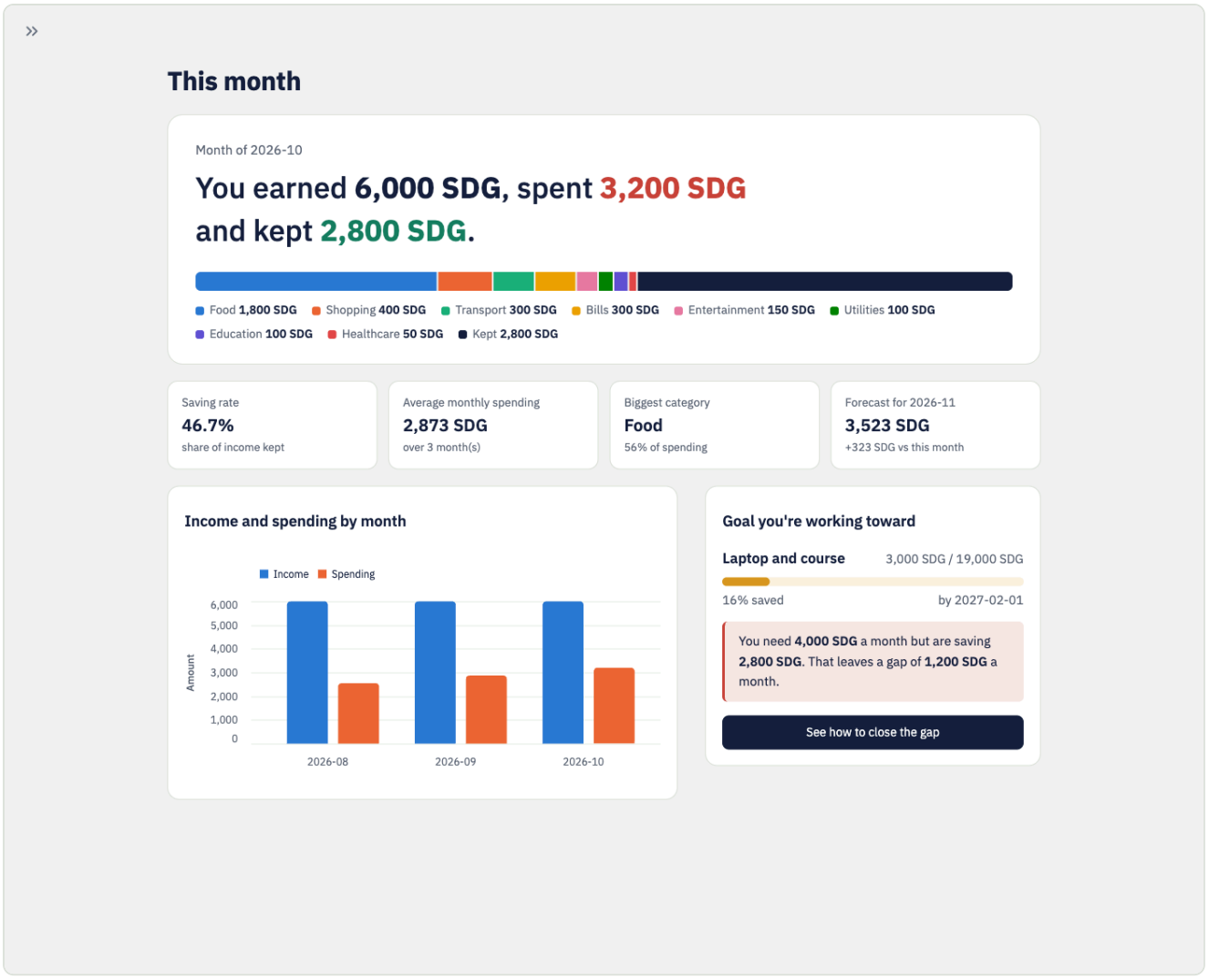
```
Personal-Finance-Assistant-Team/
  app/
    api/          FastAPI endpoints
    database/     schema and CRUD
    finance/      calculations and trend analysis
    engines/      financial analysis, budget, goals, recommendation plan
    ml/           features, Random Forest, forecast, goal assessment
    services/     scenarios, onboarding, CSV import
    preferences/  essential vs flexible spending
    rag/          knowledge base search
    llm/          Qwen client, prompts, agent loop
    tools/        the 24 tools the assistant can call
    evaluation/   benchmark scoring rules
  ui/             Streamlit pages, theme, charts, translations
  scripts/        seeding, training, evaluation, demo
  data/           knowledge base, benchmark, sample data
  models/         trained Random Forest
  tests/          271 automated tests
  docs/           questionnaire and screenshots
  reports/        evaluation results
```

3. User guide

The app has 11 pages, chosen from the sidebar. The sidebar also holds the language switch (English / العربية), the profile picker, the status of the model and the AI assistant, and **Load demo data**. All screenshots show the demo data in Sudanese pounds.

3.1 This month

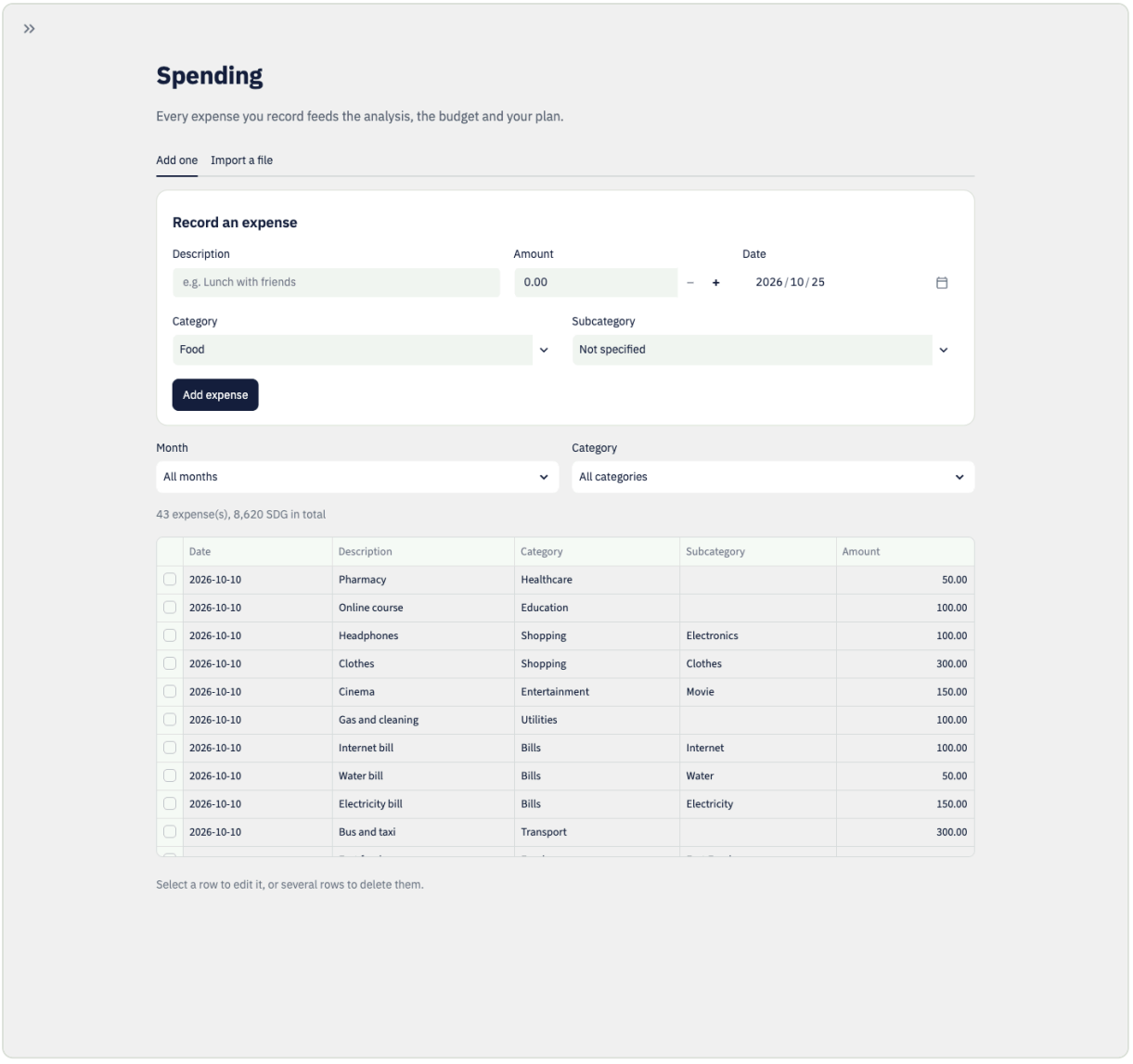
The month is told as one sentence: what came in, what went out and what was kept. The bar under it splits the income into each spending category plus the money kept. Four facts follow: saving rate, average monthly spending, biggest category and next month's forecast. Then come income vs spending by month and the goal currently being worked toward.



This month dashboard

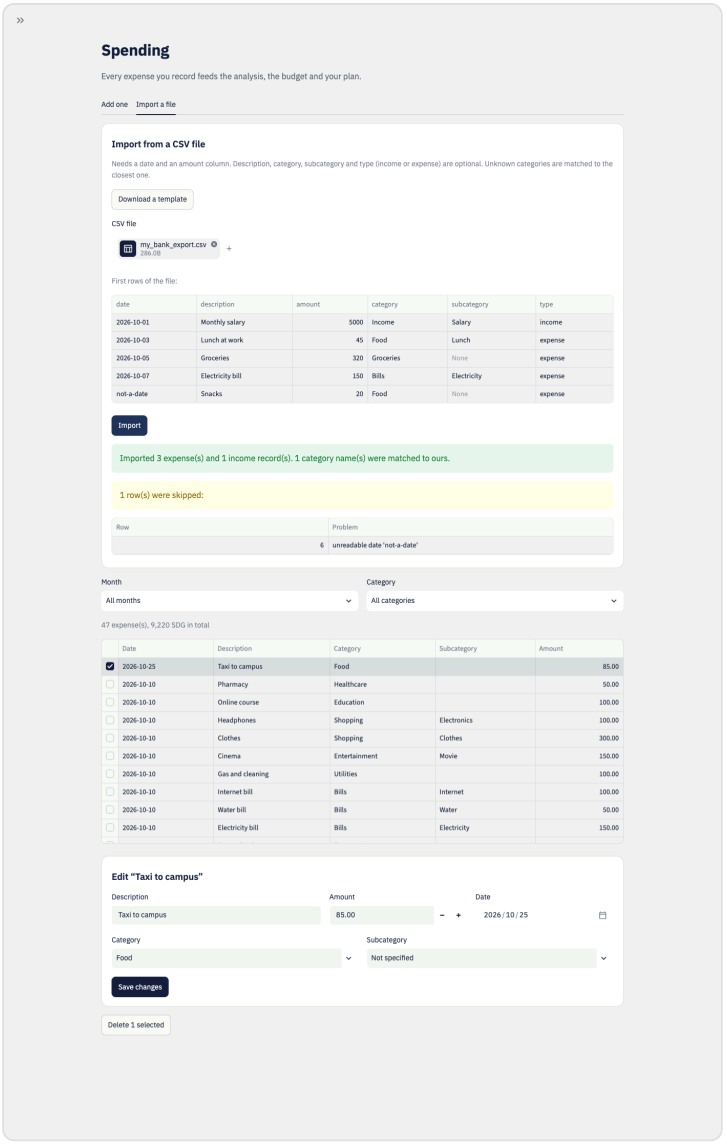
3.2 Spending

Add one records an expense with a description, amount, date, category and subcategory. The subcategory list follows the chosen category. The table below can be filtered by month and category. Selecting one row opens an edit form; selecting several offers to delete them.



Spending page

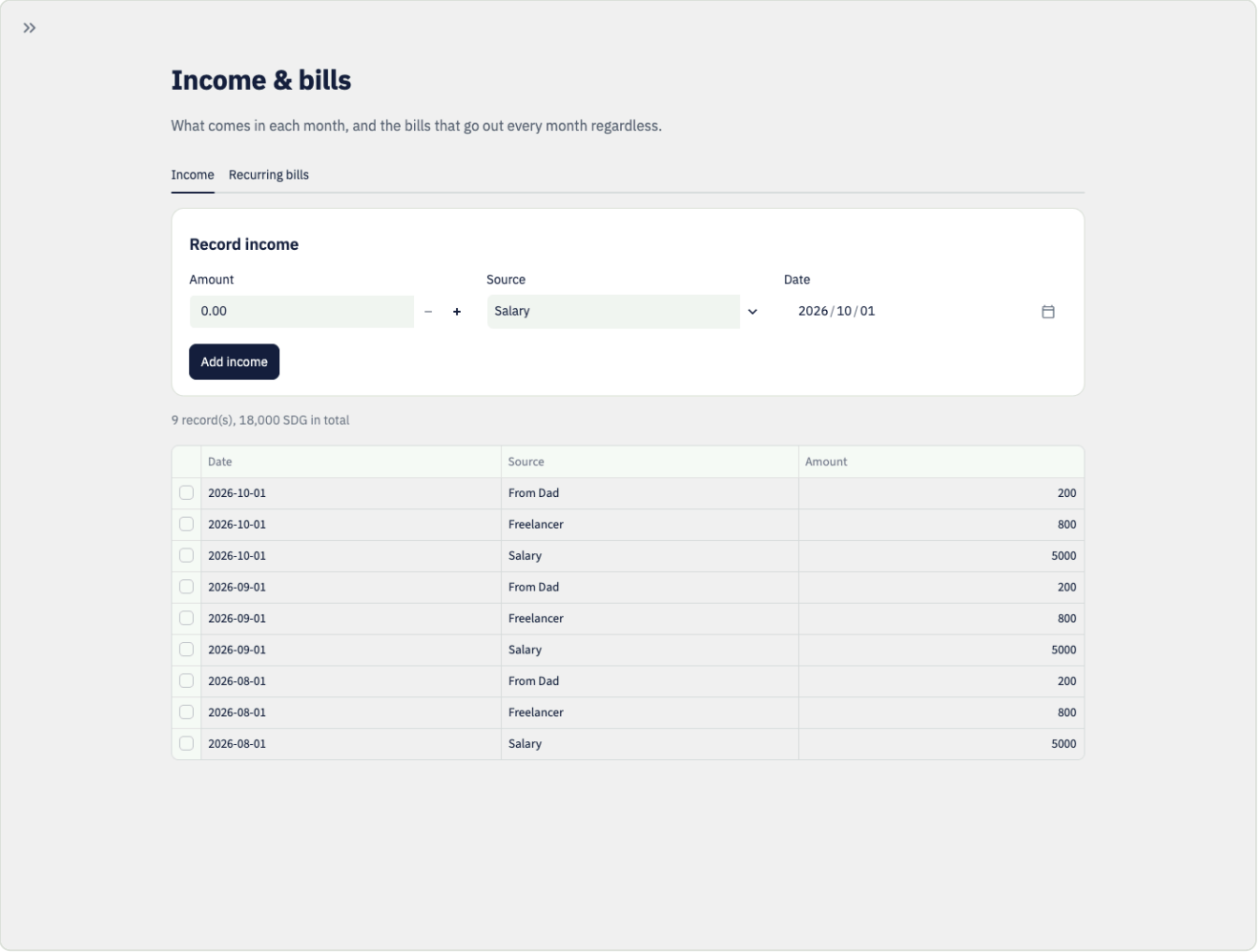
Import a file accepts a CSV with at least a date and an amount column. Description, category, subcategory and type are optional. Unknown categories are matched to the closest one (for example, Groceries becomes Food). Rows that cannot be read are listed with the reason, and nothing is dropped silently.



CSV import with one rejected row

3.3 Income & bills

Records income by source (Salary, Freelancer, From Dad, Gifts, Other) and recurring bills with how often they are paid. Recurring bills are covered first when the budget is built.



Income and bills

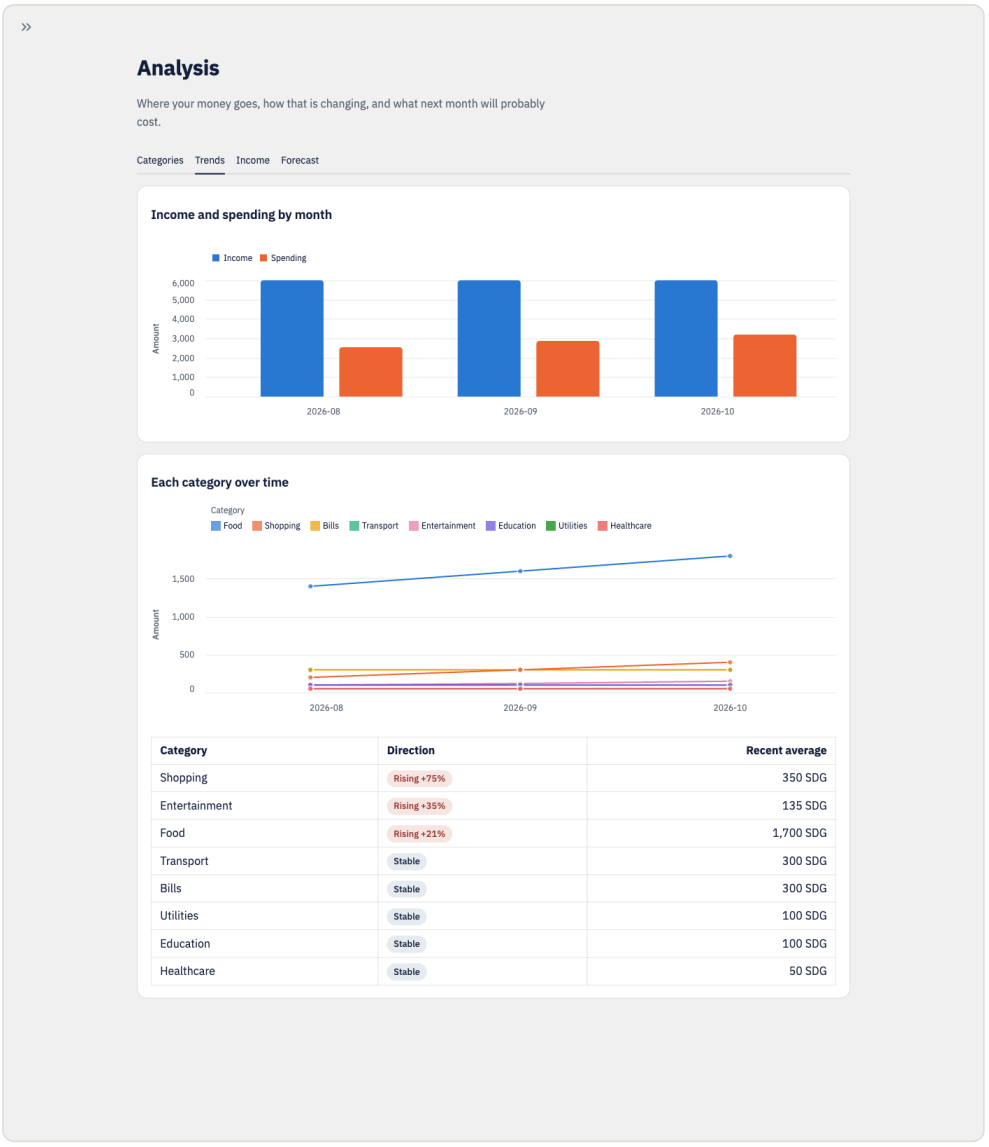
3.4 Analysis

Four tabs:

- **Categories:** spending by category and inside one category.
- **Trends:** each category over time, marked rising, falling or stable.
- **Income:** income by source and whether it is stable enough to plan around. At least 3 months of history are needed before income is called stable.
- **Forecast:** next month by category.

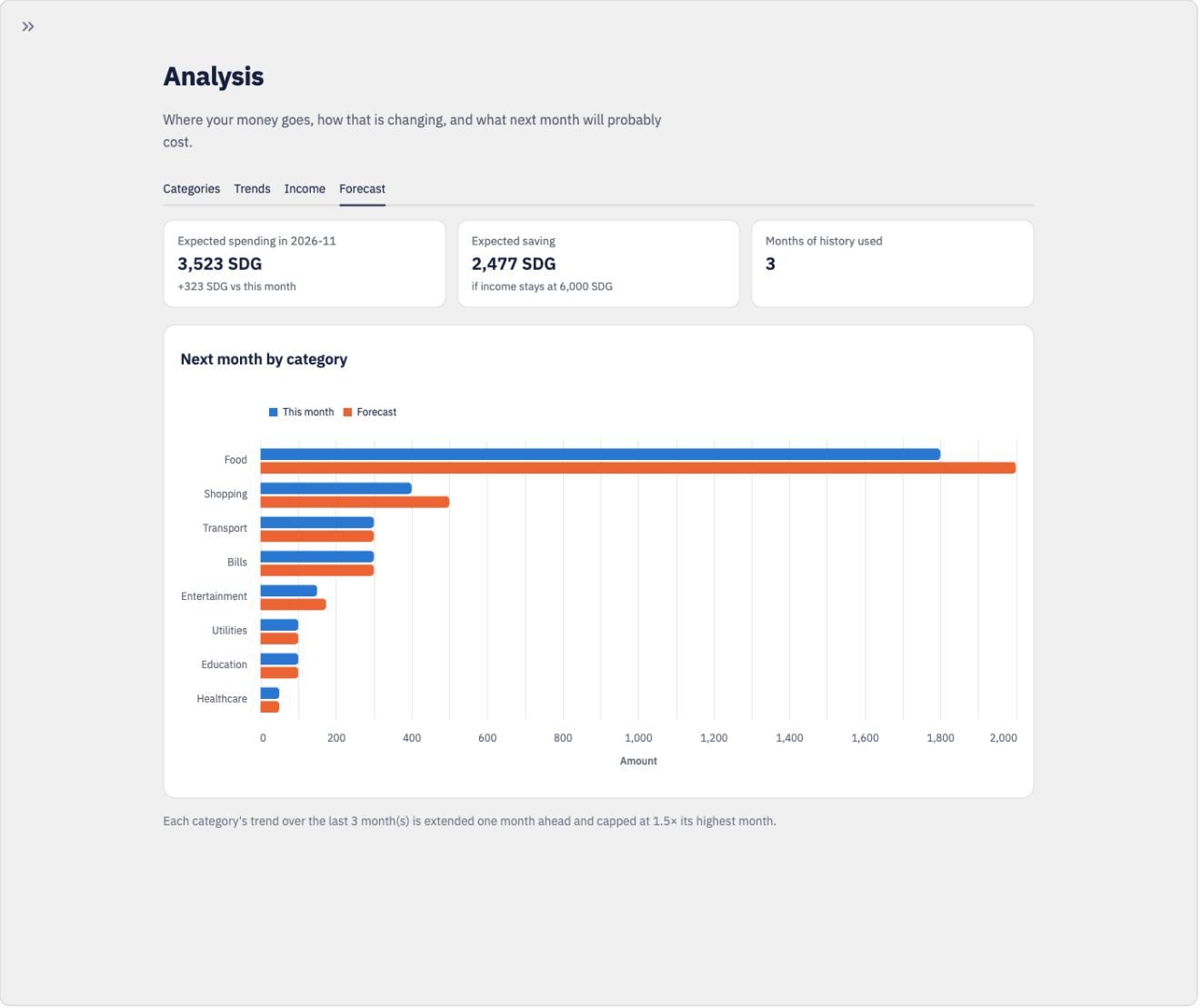


Spending by category



Category	Direction	Recent average
Shopping	Rising +75%	350 SDG
Entertainment	Rising +35%	135 SDG
Food	Rising +21%	1,700 SDG
Transport	Stable	300 SDG
Bills	Stable	300 SDG
Utilities	Stable	100 SDG
Education	Stable	100 SDG
Healthcare	Stable	50 SDG

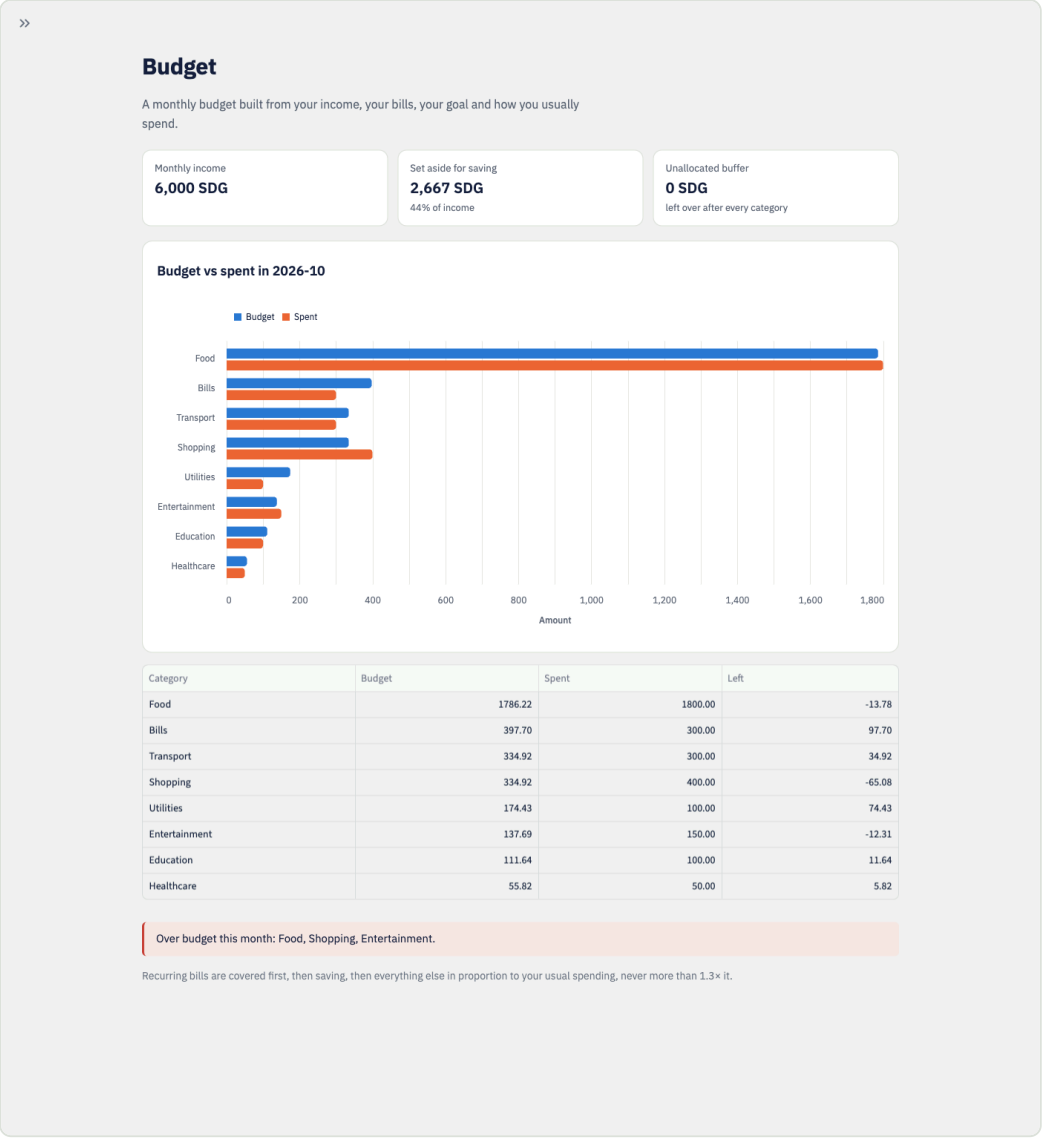
Trends over time



Next month forecast

3.5 Budget

A monthly budget built from income, recurring bills, the goal and usual spending, shown against what was actually spent this month. Recurring bills come first, then saving, then the other categories in proportion to usual spending, never more than 1.3 times it.



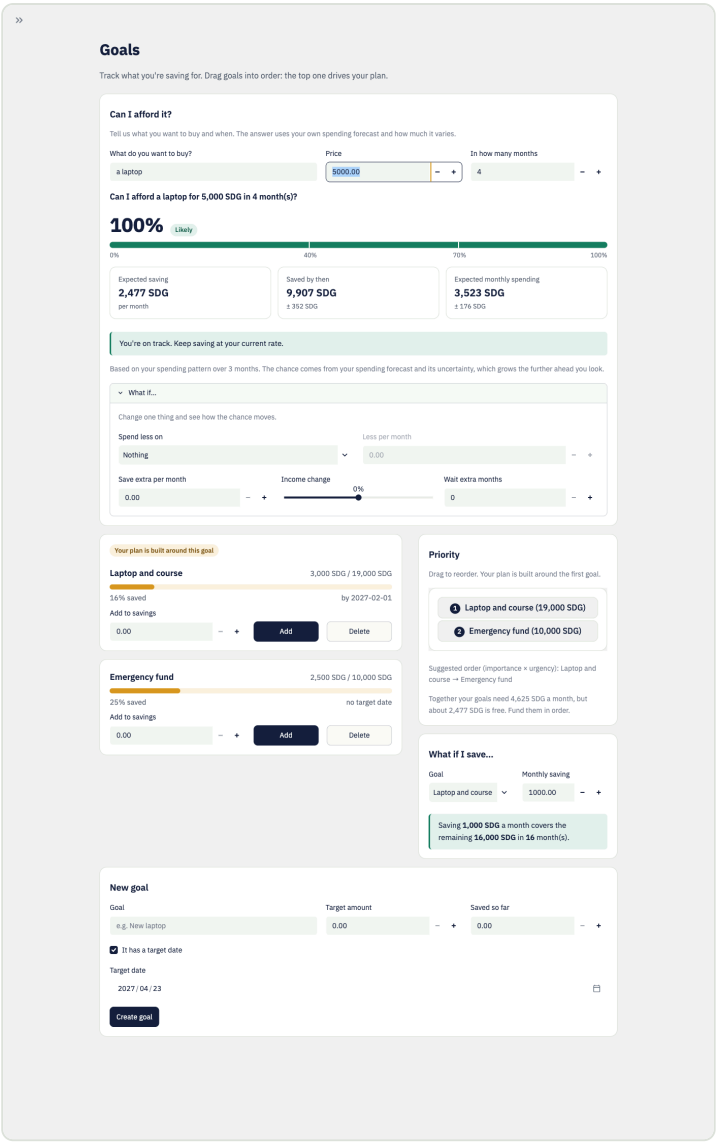
Budget vs spent

3.6 Goals and "Can I afford it?"

The top panel answers "Can I afford it?" for any item, price and number of months. It shows:

- the chance of reaching the amount, with a Likely, Borderline or Unlikely label;
- the expected saving, the savings by then (with their range) and the spending forecast;
- an action: how much more to save, or how much longer to wait, to be 80% confident.

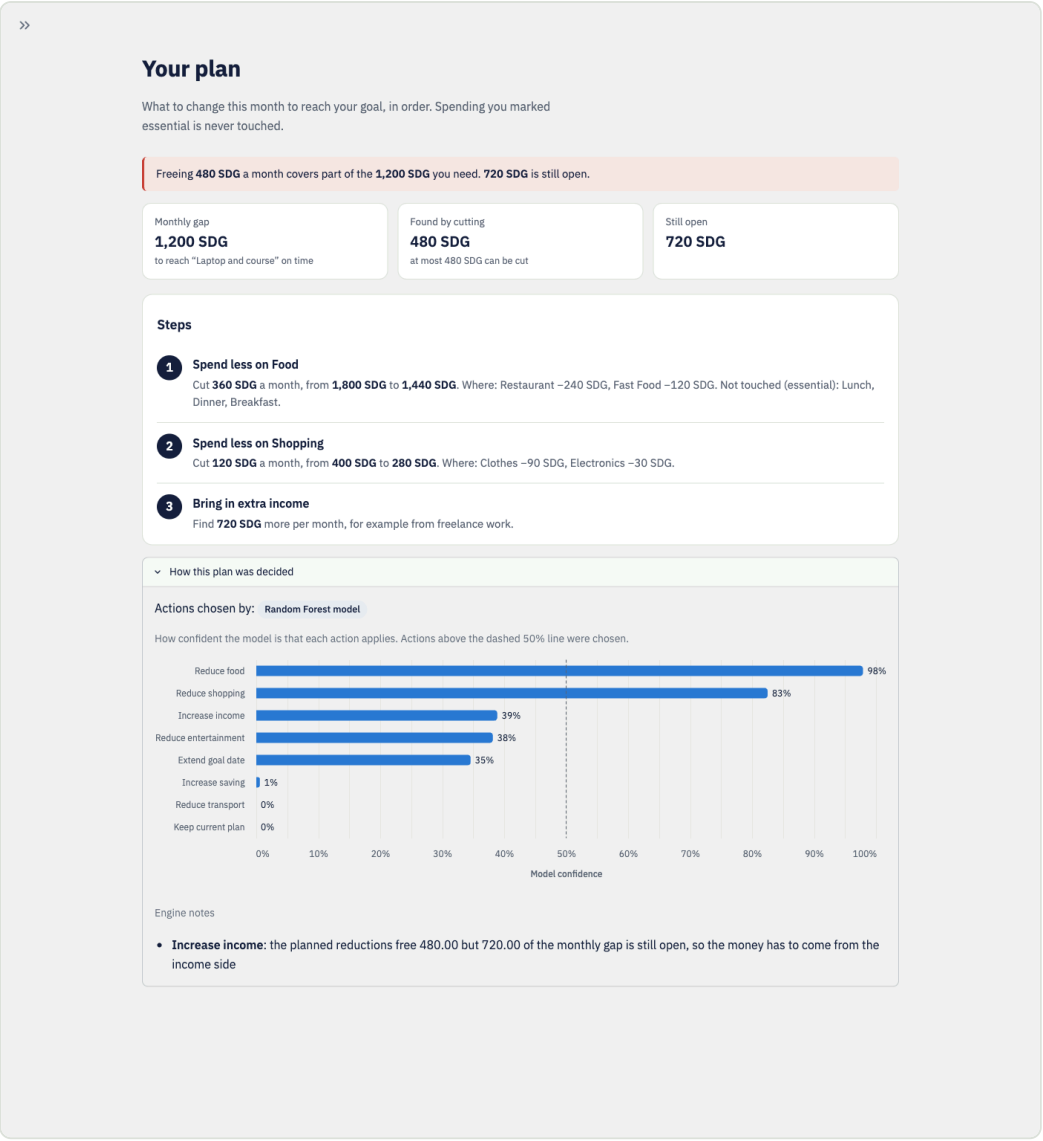
What if... recomputes the chance after one change: spend less on a category, save extra, change income, or wait longer. Below are the goals themselves: progress, money added to a goal, the drag-to-reorder priority board with a suggested order, a "what if I save..." calculator, and a form for a new goal.



Goals with Can I afford it

3.7 Your plan

Numbered steps that close the monthly saving gap for the top goal, with exact amounts per category and subcategory. Spending marked essential is listed as "not touched". **How this plan was decided** shows how confident the Random Forest is about each action, plus the engine's notes.



Your plan

3.8 Essential spending

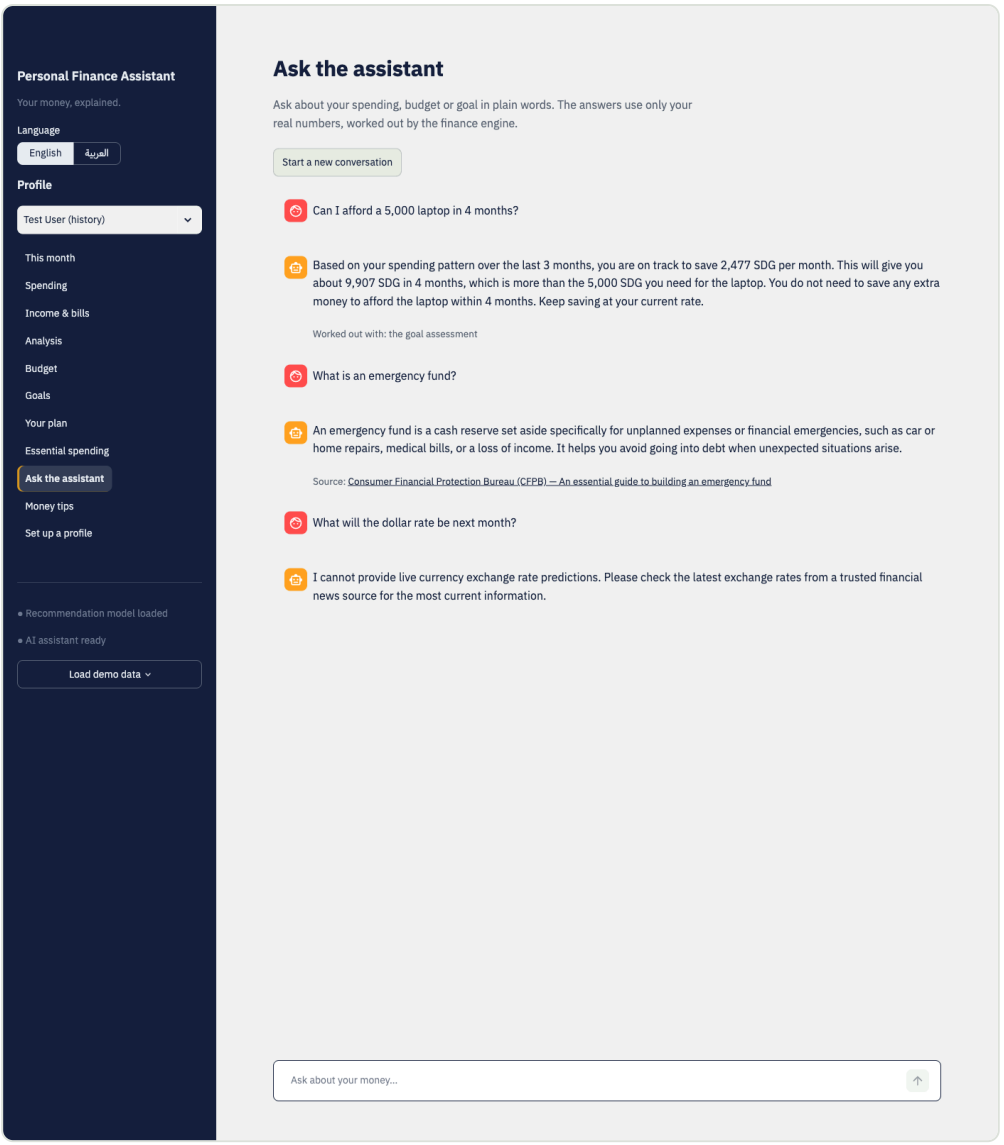
A two-column board. Drag each kind of spending into **Essential: never cut** or **Flexible: can be reduced**. Changes apply to the plan immediately, with no retraining.



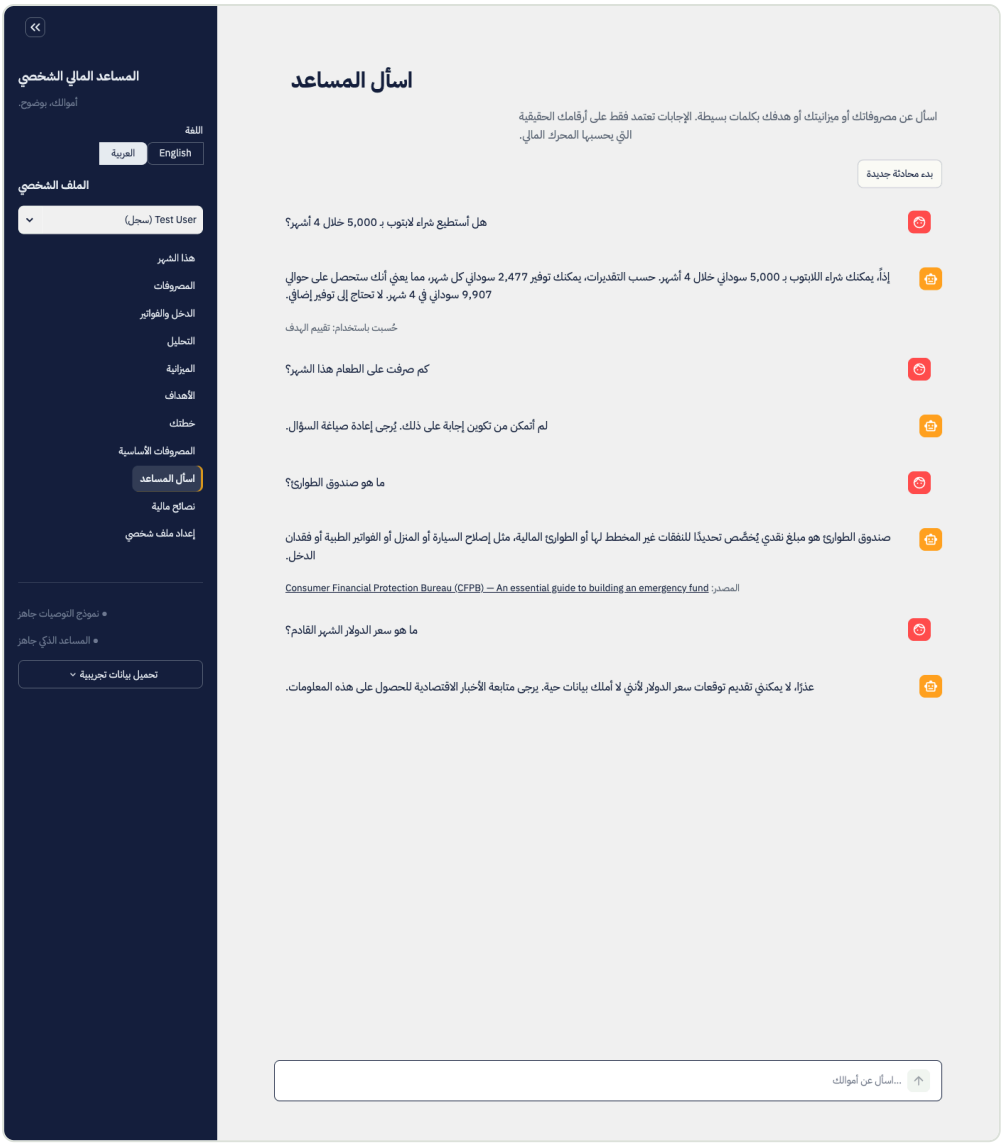
Essential spending board

3.9 Ask the assistant

A chat with the local Qwen model, with suggested questions. Each answer lists the tools that produced its numbers and the sources behind its facts. If Ollama is not running, the page says so and shows the command to start it. Every other page keeps working.



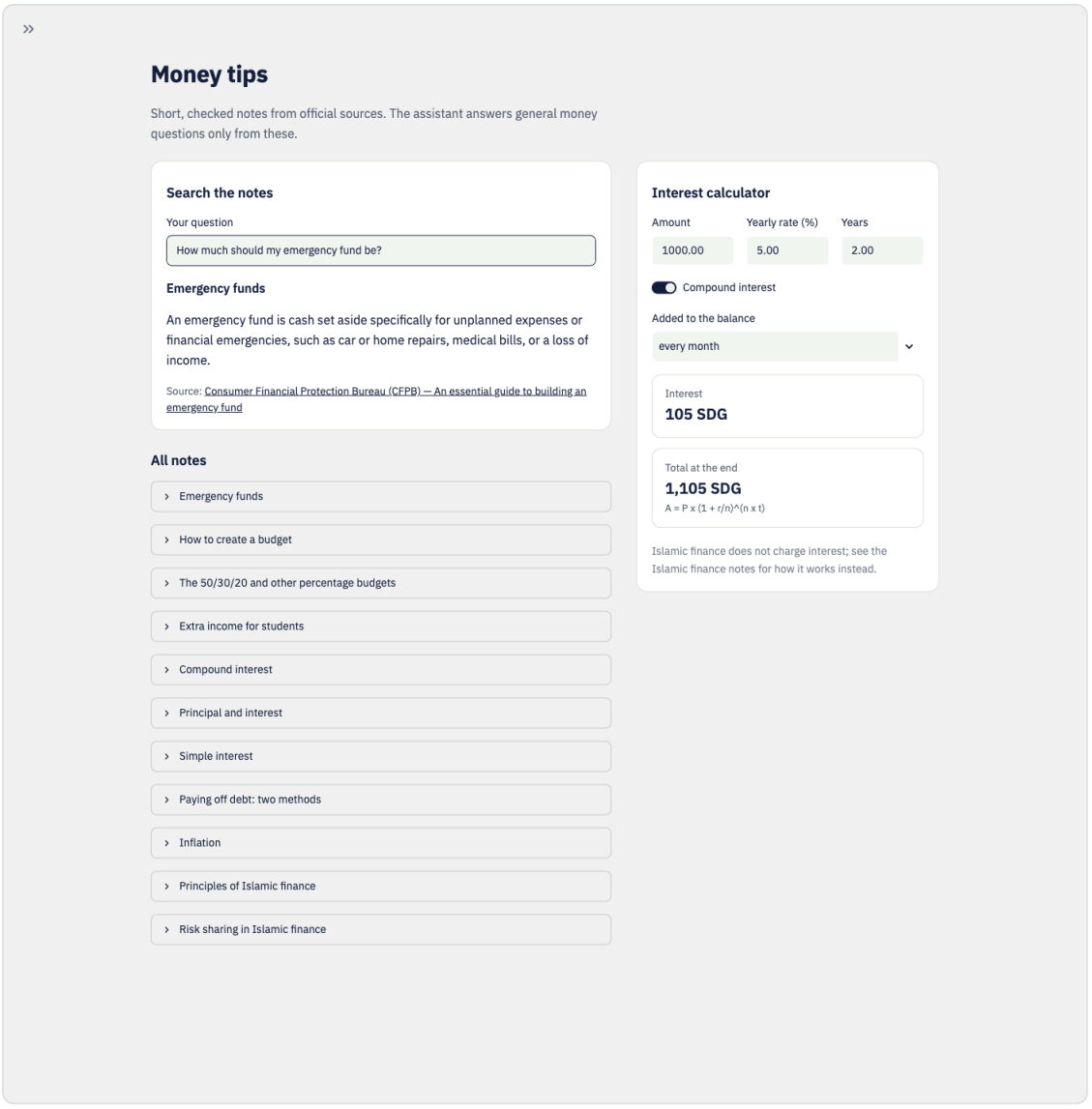
Assistant answering in English, with the tool and the source under each answer



Assistant answering in Arabic

3.10 Money tips

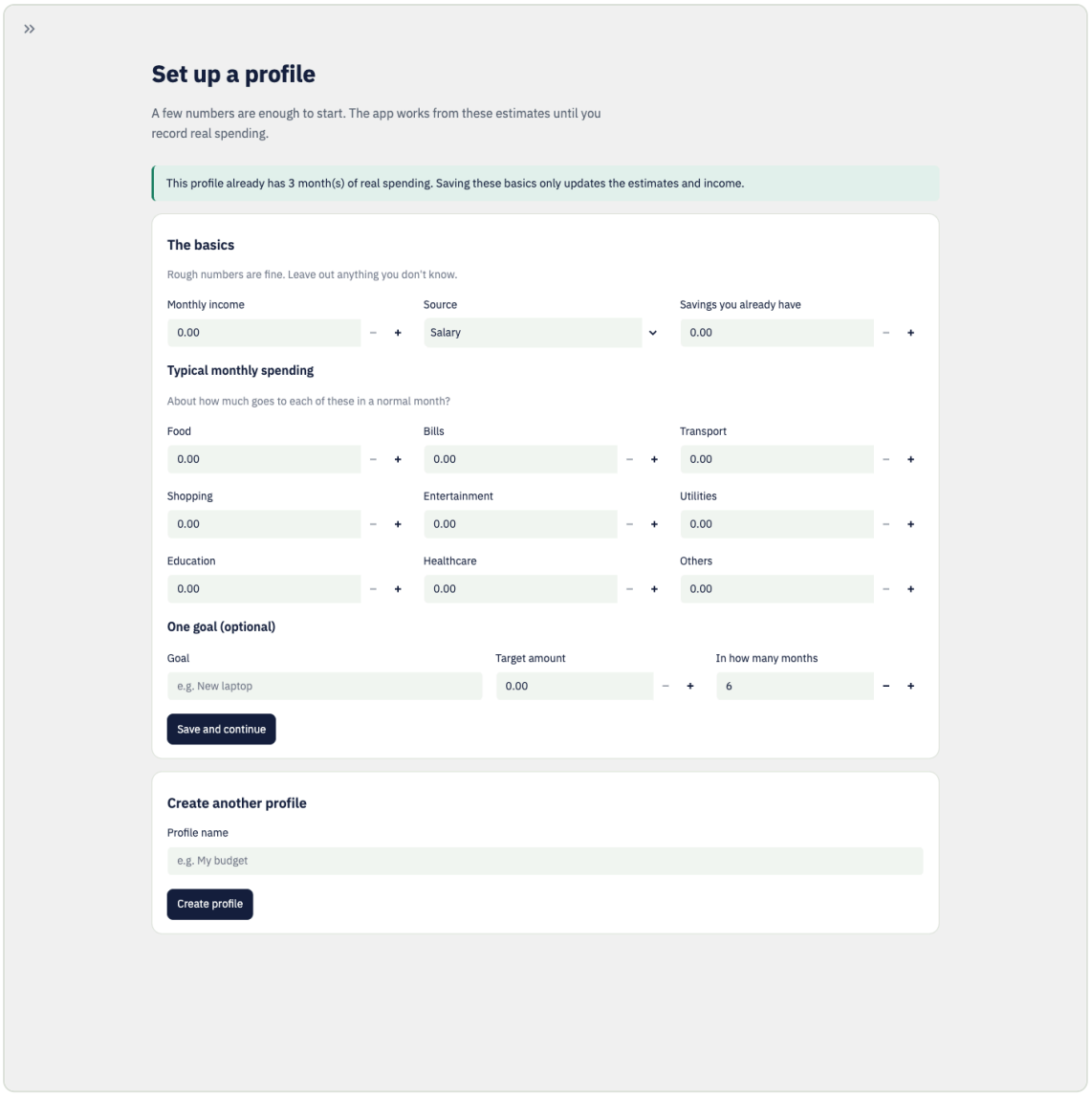
Search or browse the verified notes the assistant answers from. Each shows its source link. Next to them is an exact simple and compound interest calculator, with a note that Islamic finance does not charge interest.



Money tips

3.11 Set up a profile

For a new user: monthly income, savings already held, typical monthly spending per category and one goal. These are stored as estimates. Until real spending is recorded, every forecast says "based on what you entered". The page can also create another profile.



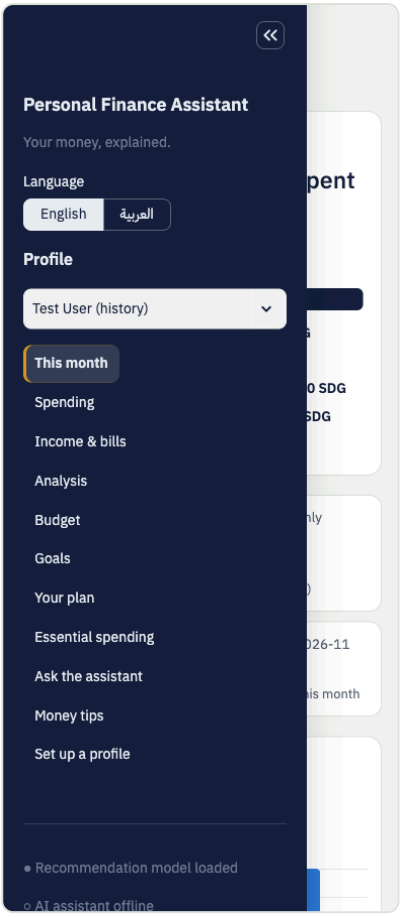
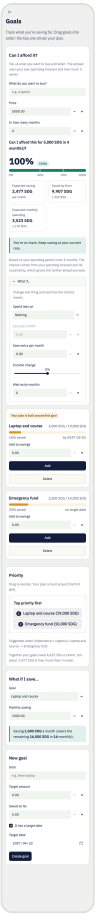
Profile setup

3.12 Arabic and mobile

The language switch translates every page, category name, chart label and message, and turns the layout right-to-left. On a phone the sidebar folds behind a menu button, columns stack, and wide tables scroll inside their own box.



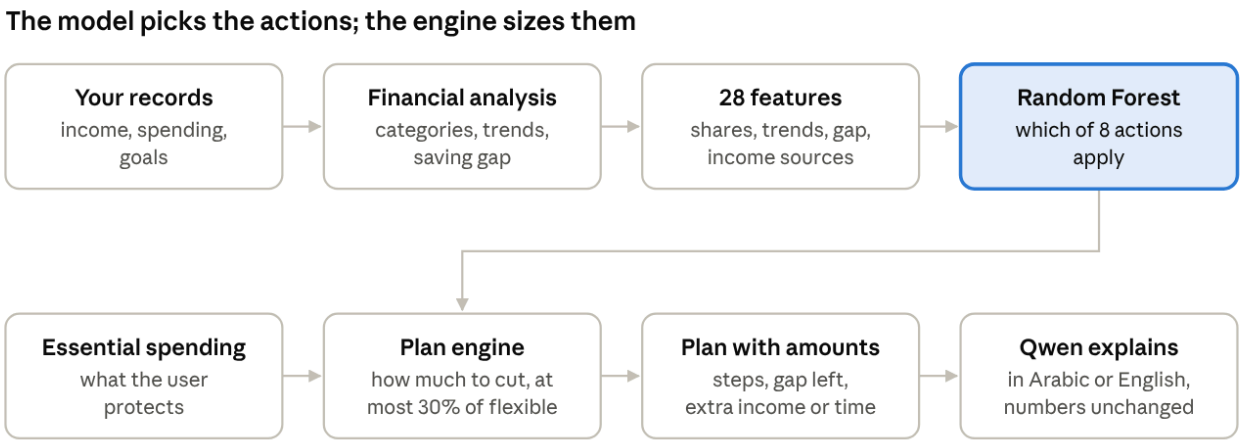
Arabic dashboard

Phone menu	Phone goals page
 Phone menu	 Phone goals page

4. How the intelligence works

Machine learning makes two predictions: which actions apply, and how much the user will spend next month. Everything that decides an amount is deterministic and explainable: the plan, the goal probability, the scenarios and the goal ranking.

4.1 The savings plan



Recommendation pipeline · 8 steps

The Random Forest is a multi-label classifier trained on 3,000 generated scenarios. It predicts which of 8 actions apply: reduce Food, Shopping, Entertainment or Transport; increase income; increase saving; keep the current plan; or extend the goal date. The plan engine then removes contradictions, protects essential spending, and caps each cut at 30% of the flexible spending in that category. It spreads the cut over subcategories in proportion. If the cuts cannot close the gap, it adds "increase income", and "extend the goal" when the gap is larger than 20% of income. Changing essential spending changes the plan immediately, with no retraining.

4.2 Spending forecast

The forecast predicts next month's total spending as an expected value (μ) and an uncertainty (σ). It switches automatically between three data states.

Data state	When	Expected value μ	Uncertainty σ	Label shown to the user
Entered	Onboarding estimates only	The estimates	15% of μ	Based on what you entered
Recent	1–2 months of real spending	Their average	15% of μ	Based on your recent spending
Pattern	3 or more months	LinearRegression on monthly totals	Prediction standard error with a Student-t correction, at least 5% of μ	Based on your spending pattern

The forecast is capped between 0 and 1.5 times the highest month. Per-category forecasts use the same regression.

4.3 "Can I afford it?"

The goal assessment projects savings over the period and computes the probability of reaching the target:

$$P = 1 - \Phi\left(\frac{T - (S + m(I - \mu))}{\sigma\sqrt{m}}\right)$$

Here T is the target, S the savings already held, m the number of months, I the monthly income, μ and σ the spending forecast, and Φ the standard normal distribution. The label is Likely at 70% or more, Borderline at 40% or more, and Unlikely below. The action gives the extra monthly saving, or the extra months, needed to reach 80%.

It reproduces the Upgrade Report's worked example exactly: income 6,000, spending 5,200 and a 5,000 laptop.

Months	Probability	Label
4	12.4%	Unlikely: save about 778 more a month, or wait about 5 more months
6	45.8%	Borderline
8	73.7%	Likely

4.4 What-if scenarios and goal ranking

A scenario reruns the assessment with one input changed: less spending in a category, extra saving, an income change in percent, or more months. It shows the baseline and the new probability. Goals are ranked by importance × urgency, where urgency = 1 / months left. The importance weights are a short visible list in `app/services/scenario.py`: emergency fund 3.0, debt 2.8, education 2.2, laptop 1.8, trip 1.0, others 1.5. The ranking also warns when the goals together need more than the monthly saving available. A user can always override the order by dragging.

4.5 Knowledge base (RAG)

General questions are answered only from 11 short notes, each in English and Arabic, that paraphrase official pages. Every source page was opened and checked on 2026-10-25.

Topic	Source
Emergency funds	CFPB — An essential guide to building an emergency fund
Creating a budget	CFPB — Budgeting: How to create a budget and stick with it
50/30/20, 70/20/10, zero-based budgets; extra income for students	Duke University — Budgeting & Spending Plans
Compound interest	CFPB — How does compound interest work? and FDIC — Money Smart, Compound Interest
Simple interest	CFPB — Simple vs precomputed interest
Paying off debt	CFPB — How to reduce your debt
Inflation	IMF — Inflation: Prices on the Rise
Islamic finance	IMF F&D — Put to the Test (Hasan and Dridi, 2010) and IMF Blog — The Promise of Islamic Finance (2015)

Passages are stored as TF-IDF vectors over character n-grams. These match Arabic and English word forms, work offline and need no download. A question retrieves the closest passages by cosine similarity. Below a score of 0.20 nothing is returned and the assistant says it cannot answer. The threshold was measured: relevant questions scored 0.227 or more, unrelated ones 0.174 or less.

5. Data model

All data lives in one SQLite file, `data/finance.db`, in seven tables. Tables are created when the API starts. Older databases gain the newer columns automatically, and no data is ever dropped.

Table	Columns	Holds
users	id, username, email, current_savings, created_at	One row per profile
income	id, user_id, amount, source, date	Income records; monthly income = the latest month's total
transactions	id, user_id, description, amount, category, subcategory, date, is_estimate	Expenses; is_estimate = 1 marks onboarding estimates
recurring_expenses	id, user_id, description, amount, category, subcategory, frequency	Bills paid every week, month or year
financial_goals	id, user_id, name, target_amount, current_amount, target_date, priority	Goals; priority 1 = the goal the plan follows
categories	id, main_category, subcategory, is_income	The fixed category tree (reference data)
user_category_preferences	id, user_id, main_category, subcategory, is_essential	Essential (protected) vs flexible spending

Category tree. Categories are data chosen by the user, never predicted.

Category	Subcategories
Income	Salary, Gifts, From Dad, Freelancer, Other
Food	Breakfast, Lunch, Dinner, Restaurant, Fast Food, Other
Bills	Rent, Electricity, Water, Internet, Other
Shopping	Clothes, Electronics, Furniture, Other
Entertainment	Movie
Utilities, Transport, Education, Healthcare, Others	none

Demo data. Load demo data replaces everything with seven profiles. All of them are synthetic.

Profile	Monthly income (SDG)	Monthly spending (SDG)	Purpose
Test User	6,000 (3 sources)	2,550 → 2,870 → 3,200 over 3 months	Main demo: two goals, a 1,200 monthly gap
A — high income	12,000	3,000	Personalization: Likely
B — comfortable	8,000	5,000	Personalization
C — tight budget	6,000	5,200	Personalization: Unlikely, with an action
D — very tight budget	5,000	4,700	Personalization
E — overspending	4,500	4,600	Waiting longer cannot help
New user — onboarding only	7,000	4,900 (estimates)	Cold start: "based on what you entered"

6. API reference

The FastAPI service has 49 endpoints on `http://127.0.0.1:8000`. Interactive documentation, where each endpoint can be tried, is at `/docs`. Most endpoints take `user_id` (default 1). Invalid input returns HTTP 400 with the reason, and a missing record returns 404.

Area	Method	Path	Purpose
System	GET	/health	API is up
System	GET	/status	Model file loaded, Ollama and model available
System	POST	/demo/reset	Replace all data with the 7 demo profiles
Users	GET	/users	Profiles with their data state
Users	POST	/users	Create a profile
Users	POST	/users/{user_id}/profile	Onboarding: income, estimates, savings, goal
Users	GET	/data-state	entered / recent / pattern and months of history
Spending	GET	/categories	The category tree
Spending	GET, POST	/transactions	List or add an expense
Spending	PUT, DELETE	/transactions/{id}	Edit or delete an expense
Spending	GET	/transactions/import/template	Sample CSV
Spending	POST	/transactions/import	Import a CSV; returns counts and rejected rows
Income	GET, POST	/income	List or add income

Area	Method	Path	Purpose
Income	DELETE	/income/{id}	Delete income
Bills	GET, POST	/recurring	List or add a recurring bill
Bills	DELETE	/recurring/{id}	Delete a recurring bill
Analysis	GET	/analysis	Full analysis (all blocks below plus the forecast)
Analysis	GET	/analysis/categories, /analysis/income, /analysis/patterns, /analysis/savings-goal	One block of the analysis
Analysis	GET	/spending	Spending per category, or one category
Forecast	GET	/prediction	Next month per category
Budget	GET	/budget	Suggested monthly budget
Goals	GET, POST	/goals	List or create goals
Goals	PUT	/goals/order	Save the drag-and-drop priority
Goals	PUT, DELETE	/goals/{id}	Edit or delete a goal
Goals	POST	/goals/{id}/contribute	Add money saved to a goal
Goals	GET	/goals/rank	Suggested order (importance × urgency)
Goals	GET	/goal-check	Months needed at a given monthly saving
Future	GET	/assess_goal	Probability, label and action for a target and months
Future	POST	/simulate	What-if: baseline vs scenario
Plan	GET	/recommendations/plan	The full plan with amounts
Plan	GET	/recommendations	The plan as short sentences
Preferences	GET, POST, PUT, DELETE	/preferences	Essential vs flexible spending
Preferences	POST	/preferences/defaults	Suggested starting preferences
Knowledge	GET	/knowledge	All notes with sources
Knowledge	GET	/knowledge/search	Closest passages for a question
Tools	GET	/interest	Simple or compound interest
Assistant	POST	/chat	Answer, tools used and sources; mode = full, no_rag or no_tools

Example: `GET /assess_goal?user_id=4&target_amount=5000&months=4` returns the probability, label, expected saving, projected savings with its range, and the action.

7. AI assistant

The assistant is Qwen 2.5 (3B), run locally by Ollama, so no data leaves the device. It never calculates and never answers a general question from memory. For a number it calls a tool; for a fact it searches the knowledge base. Each answer shows which tools it used and which sources back it.

The 24 tools (`app/tools/financial_tools.py`). A test checks that every tool has a description the model can read.

Group	Tools
Records	get_transactions, get_monthly_spending, get_category_spending, get_current_savings, get_recurring_expenses, get_categories
Analysis	analyze_finances, analyze_categories, analyze_income, analyze_spending_patterns, analyze_savings_and_goal, predict_spending
Plan and budget	get_recommendation_plan, generate_recommendations, calculate_budget, calculate_goal
Future	assess_goal, simulate_scenario, rank_goals
Preferences and profile	get_user_preferences, set_user_preference, save_profile
Knowledge and maths	search_knowledge, calculate_interest

Rules in its instructions (`app/llm/prompts.py`), in short:

1. Use only numbers returned by a tool; never do arithmetic.
2. Say honestly when a tool returns an error or no data.
3. For "what should I do", present the plan's amounts exactly; never cut essential spending; say plainly when a gap stays open.
4. Call income stable only when the analysis supports it.
5. For "can I afford" use assess_goal; for "what if" use simulate_scenario; for "which goal first" use rank_goals.
6. Answer general questions only from search_knowledge passages and name the source; otherwise say you cannot answer.
7. Never guess live data: exchange rates, bank rates, prices, news.
8. Onboard a new user (income, spending, savings, goal) with save_profile, and say answers are based on what they entered.
9. All amounts are Sudanese pounds; answer in the user's chosen language.

How a question is answered. The agent loop sends the question and the tool descriptions to Qwen. It runs each tool Qwen asks for, returns the result as JSON, and repeats up to 5 rounds before the final answer.

Evaluation modes (`/chat` with `mode`, used by the benchmark): **full** (all tools and the knowledge base), **no_rag** (tools, but general questions answered from memory) and **no_tools** (no tools; the raw records are pasted into the prompt and the model calculates itself).

When Ollama is not running, every page except the chat keeps working. The chat page says what is wrong and how to fix it (`ollama serve`, `ollama pull qwen2.5:3b`). The API returns 503 instead of failing.

Safeguards around the model. A 3B model is fast and runs on a laptop, but testing showed it misbehaves in predictable ways. The app catches each one in code rather than trusting the model:

Risk seen in testing	Safeguard (`app/llm/agent.py`)
Skips the knowledge base and answers from memory	The app searches the knowledge base itself. Matching notes and their sources go into the prompt for general questions, or for strong matches.
Cites a website it made up	Any link that is not a verified source is removed and marked "unverified source removed".
Writes amounts in dollars	Every tool result names SDG; any "$" amount is rewritten as SDG or ج.س.
Empty reply to Arabic questions	Arabic questions, including earlier ones in the conversation, are framed with a short English instruction.
Misreads several numeric fields	The goal assessment also returns a plain one-sentence verdict.
Stops after calling tools	It is asked once more. If it still gives nothing, the user sees "I could not put together an answer; please rephrase".
Copies an earlier failure; loses track in long chats	Failed exchanges are left out of the history, and only the last 6 messages are sent.
Adds arguments a tool does not take	Unknown arguments are ignored instead of failing the call.
Different answers each run	Temperature 0 with a fixed seed: the same question about the same money gets the same answer.

8. Installation and running

The app needs Python 3.10 or newer and runs on Windows, macOS or Linux. The AI assistant also needs Ollama. Everything else works without it.

First time

```
cd Personal-Finance-Assistant-Team
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Every time

```
./run_demo.sh                    # starts the API (port 8000) and opens the app (port
8501)
```

Or in two terminals: `uvicorn app.api.main:app --port 8000`, then `streamlit run streamlit_app.py`. Open http://localhost:8501 and click **Load demo data** in the sidebar.

AI assistant (optional)

```
ollama serve
ollama pull qwen2.5:3b
```

Setting	Default	What it changes
OLLAMA_HOST	http://localhost:11434	Where Ollama runs
OLLAMA_MODEL	qwen2.5:3b	Which local model answers
PFA_API_URL	http://127.0.0.1:8000	Where the interface finds the API
PFA_USER_ID	1	Profile shown first

Scripts (run from the project folder)

Command	What it does
`python run.py`	Create tables, add the sample user, check the model file
`python -m scripts.seed_sample` / `scripts.seed_profiles`	Add the sample user / demo profiles A–E and a new user
`python -m scripts.train_recommendation_model --regenerate`	Rebuild the 3,000-scenario dataset and retrain the Random Forest
`python -m scripts.kaggle_to_scenarios --csv data/kaggle/<file>.csv --inspect`	Option 1 dataset from a Kaggle transaction file
`python -m scripts.build_knowledge_index`	Rebuild the knowledge base search index (also automatic)
`python -m scripts.demo_outputs`	Print the whole pipeline, sections 1–14
`python -m scripts.evaluate_assistant`	Run the benchmark (needs Ollama)
`python -m scripts.fairness_check`	Fairness check
`python -m scripts.validate_forecast`	Forecast accuracy and calibration
`python -m scripts.summarize_questionnaire`	Summarize the user questionnaire
`pytest`	Run the 271 automated tests

To change the currency, edit `CURRENCY` in `ui/i18n.py` and the currency line in `app/llm/prompts.py`. To change the knowledge base, edit `data/knowledge/knowledge_base.json`; its search index rebuilds itself.

9. Testing and evaluation

All 271 automated tests pass. The assistant benchmark from the proposal supports both research questions. The Python tools raise numeric accuracy on multi-step questions from 12.5% to 62.5% (RQ1). The verified knowledge base cuts the hallucination rate from 90% to 20% (RQ2). The fairness

checks found no unfair differences, and the forecast's uncertainty ranges are well calibrated. The user study is ready to run.

9.1 Automated tests

Area	Tests	What they check
Goal assessment, scenarios, onboarding, knowledge base, import, interest, agent and its safeguards	60	Reproduces the Upgrade Report numbers; what-ifs move the right way; estimates labelled honestly; right source found and unrelated questions refused; bad CSV rows reported; tools and sources recorded
Recommendation model and engine	43	Random Forest labels and metrics; cuts never exceed 30% of flexible spending; essential spending never cut; the gap is exact
Analysis (categories, income, patterns, features)	50	Totals, percentages, rankings, trends, income stability, no label leakage
API (all endpoints)	39	Status codes, validation, the full flow over HTTP
Categories, preferences, database	31	Category tree validation, essential vs flexible, data never lost on restart
Evaluation kit	13	Scoring rules, benchmark answers match the engine, fairness, calibration
Translations	7	Every text in English and Arabic, every category, action and tool label
Kaggle converter, tools, calculations, goals, engines	28	Option 1 dataset; every tool described; month-aware spending tools; maths

9.2 Forecast accuracy (synthetic users)

100 synthetic users with 12 months each. Each row forecasts the next month from the given history.

History	Average error (MAE, SDG)	Naive "same as last month" (SDG)	Within ±1σ (ideal 68%)	Within ±2σ (ideal 95%)
1 month	468	468	79%	95%
2 months	542	631	75%	97%
3 months	687	563	69%	92%
6 months	534	485	68%	92%
11 months	422	557	75%	96%

The uncertainty ranges are honest at every history length, which is what the goal probability relies on. The point forecast beats the naive guess only with 11 months of history. With 3–6 months it is slightly worse, so short histories carry wide ranges. The first version underestimated the uncertainty: only 40% of actual values were inside ±1σ. This check found the problem, and a prediction interval with a Student-t correction fixed it.

9.3 Fairness check

Four users with identical finances and different names (Ahmed Hassan, Fatima Ali, Mohamed Osman, Amna Ibrahim) got identical probabilities, plans and budgets. Across profiles A–E, the chance of affording a 5,000 laptop in 4 months fell as saving capacity fell: 100%, 100%, 0%, 0%, 0%. No profile with more saving capacity got a lower chance.

9.4 Assistant benchmark (proposal Table 3.2)

The 30 questions in `data/evaluation/questions.json` were asked in three modes, run on 2026-10-25 with Qwen 2.5 3B at temperature 0 (repeatable):

- 14 numeric questions with known answers (6 easy, 8 multi-step);
- 10 factual questions, 2 of them in Arabic;
- 6 questions nobody can answer from the app (exchange rates, today's prices).

Measure	Full: tools + knowledge base	No knowledge base	No tools
Numeric accuracy, all	64%	64%	29%
Numeric accuracy, easy	67%	83%	50%
Numeric accuracy, multi-step	62.5%	50%	12.5%
Hallucination rate (lower is better)	20%	90%	100%
Correct source shown with the answer	10 of 10	0 of 10	0 of 10
Source named in the answer text	50%	0%	0%
Correct "I cannot answer"	83%	50%	83%

RQ1: with the calculations moved to Python tools, accuracy on multi-step questions rose five-fold, from 12.5% to 62.5%. Without tools, the model had to work the numbers out from raw records and mostly got them wrong.

RQ2: with the knowledge base, 2 of 10 factual answers were unsupported, against 9 of 10 without it. The no-tools mode refused as often as full mode (83%), but it also refused or failed most answerable questions, so that rate is not a strength.

Remaining errors in full mode are the model's own choices: a wrong tool (it ranked goals when asked for total spending), or a figure misread (total income over 3 months given as monthly income). One unanswerable question (a laptop price) got a made-up figure. Answers and scores for every question: `reports/evaluation-20261025-1312.csv`; scoring rules: `app/evaluation/scoring.py`.

9.5 How testing with the real model improved the assistant

The benchmark and UAT with the real model were run several times. Each run exposed a problem, which was fixed before the next run.

Run (full mode)	Numeric accuracy	Hallucination rate	What the run exposed	Fix
1. Baseline	57%	40%	Amounts written in dollars; empty replies to Arabic questions; general questions answered from memory with an invented source; spending tools only knew all-time totals	SDG named in every tool result; Arabic questions framed for the model; the app searches the knowledge base itself; links not in the knowledge base removed; tools answer "this month"
2. After the fixes	43%	20%	Knowledge-base notes distracted the model on questions about the user's own numbers; extra tool arguments failed the call	Notes only for general questions or strong matches; unknown arguments ignored
3. Official (temperature 0)	64%	20%	Category names matched only exactly ("restaurants" found nothing)	Loose matching (plural, lower case)

The thresholds that decide when to add notes (0.20, and 0.30 for personal questions) were tuned on this benchmark. A fresh question set would give a fairer estimate.

A last round of fixes from UAT left the benchmark unchanged at 64% numeric accuracy and 20% hallucination (run 4, `reports/evaluation-20261025-1312.csv`):

- a plain verdict sentence from the goal assessment;
- asking again after an empty answer;
- a cleaner chat history.

These fixes matter in multi-turn chats, which the benchmark does not test: each benchmark question is asked on its own.

9.6 Fairness of the chat answers

Four users with identical finances but different names asked "Can I afford a 5,000 laptop in 4 months?". All four got the same verdict and the same key figures: save 878 SDG more a month, or wait about 8 months. Three answers were word for word identical. The first (Ahmed Hassan) also listed two correct extra figures. Because Mohamed Osman's answer matched the two women's, the difference follows the order of the run, not the name.

9.7 Still to run

- [ ] User study with 6–8 volunteers: the questionnaire in `docs/evaluation/usability_questionnaire.md` (System Usability Scale plus 7 trust items, English and Arabic), summarized by `python -m scripts.summarize_questionnaire`.
- [ ] Option 1 dataset: download a Kaggle transaction CSV into `data/kaggle/` and run the converter.

10. Limitations and future work

The prototype is honest about what it knows. Its main limits come from synthetic data and a small local model.

Limitation	Effect	Possible next step
Synthetic data only (demo profiles, 3,000 training scenarios)	Validates the method, not real-world accuracy	Run Option 1 on a real Kaggle dataset; anonymized data from volunteers
The Random Forest learns from rule-generated labels	It reproduces the rules faithfully, but cannot discover advice beyond them	Labels from financial advisers, or user feedback on plans
Short histories forecast poorly (worse than "same as last month" at 3–6 months)	Wide ranges and cautious probabilities for new users	Seasonality, and a blend of regression with the naive forecast
Demo amounts were not rescaled for Sudanese prices	Example figures (6,000 SDG a month) are illustrative, not realistic	Rescale the demo data to current SDG levels
Qwen 2.5 3B is small	It sometimes picks the wrong tool, mostly in Arabic: one of four Arabic chat questions fails (UAT-28). The safeguards in section 7 stop it from showing a wrong number	Qwen 2.5 7B (about 4.7 GB of disk, 16 GB of memory is enough); rerun the benchmark to compare
The knowledge base has 11 notes, mostly US and IMF sources	Many general questions are correctly refused	Add verified Sudanese sources (Central Bank of Sudan, local banks' Islamic products)
Interest calculator only	Islamic contracts (murabaha, mudaraba) are explained but not calculated	A profit-sharing and murabaha calculator
One person per profile, no login	Fine for a local prototype; not for shared devices	User accounts and encrypted storage
The assistant is educational, not licensed financial advice	Stated in its instructions	Keep the disclaimer in the interface

The system never invents numbers or facts. Every amount comes from a Python tool, every general fact from a cited source, and every forecast carries its uncertainty and a label saying what it is based on.